Automated Calibration and Error Mitigation of a Variational Quantum Circuit for Credit Risk Analysis on Noisy Hardware

Lalith Sagar
School of Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore, India
CB.AI.U4AID24009

Ruthwik
School of Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore, India
CB.AI.U4AID24023

Muthu Raam
School of Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore, India
CB.AI.U4AID24058

Prapul Chandra
School of Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore, India
CB.AI.U4AID24063

***Abstract*—Quantum algorithms for credit risk analysis need a circuit that loads the risk model into the amplitudes of a qubit register. On present noisy quantum processors this circuit does not behave as designed, because every rotation gate is slightly mis-calibrated and the calibration drifts over time. A circuit that is exact in simulation therefore gives a wrong default probability on hardware. The proposed work builds a complete, hardware-aware pipeline for the one-factor Gaussian conditional-independence credit risk model and add four improvements on top of the published manual procedure. First, the manual grid sweep used to retune the gate angles is replaced by an automated calibration that combines Bayesian optimisation with simultaneous perturbation stochastic approximation. Second, readout-error mitigation and zero-noise extrapolation are applied to the measured data. Third, a fidelity gate accepts or rejects every quantum result. Fourth, a closed loop detects calibration drift and repairs it without human input. All experiments run on a noise-emulated backend with realistic gate errors, readout errors and hidden angle drift. Under identical shot budgets, the automated calibration reaches a Hellinger fidelity of 0.983 and 0.992 for two- and three-qubit registers, compared with 0.900 and 0.948 for the manual grid sweep, while using half the circuit executions. After a drift event the closed loop restores the fidelity above the acceptance threshold in one round. The full proposed pipeline needs 44 percent fewer circuit executions than the reference procedure and runs in the same or less wall-clock time on a laptop.**

***Index Terms*—quantum finance, credit risk, variational quantum circuit, Bayesian optimisation, SPSA, quantum error mitigation, NISQ calibration**

I. Introduction

Banks must estimate how likely a borrower is to default and how much money they could lose in a bad year. Two numbers summarise this risk: the expected loss and the value at risk (VaR), which is the loss that is not exceeded with a chosen confidence, for example 95 %. For a large portfolio these numbers are computed with Monte Carlo simulation, whose error falls only as $1/\sqrt{N}$ with the number of samples N. Quantum amplitude estimation (QAE) can estimate the same quantities with an error that falls as $1/N$, a quadratic improvement [2]–[4].

QAE needs one building block before it can be used: a quantum circuit that loads the risk model, so that the probability of measuring the "default" state equals the true default probability. Building this loading circuit is the subject of the proposed work. On current noisy intermediate-scale quantum (NISQ) processors [5] the circuit faces three practical problems.

- **Gate mis-calibration.** The processor does not rotate a qubit by exactly the commanded angle. A small over-rotation and offset on every gate distort the loaded distribution.
- **Measurement and gate noise.** Qubits are sometimes read out with the wrong value, and every two-qubit gate adds noise.
- **Drift.** The calibration of the processor changes over hours or days, so a circuit that was tuned yesterday may fail today.

The reference work by Ahmad *et al.* [1] showed that these effects can be handled by retuning the gate angles directly on the hardware with a coarse-to-fine grid sweep. The sweep is effective but manual. It tests every point of a fixed grid, tunes one angle (or one pair of angles) at a time, applies no readout correction, and gives no rule for deciding whether a result can be trusted or when the circuit must be retuned again.

The proposed work builds the complete pipeline and turns the manual retuning into an automated, closed-loop procedure. The main contributions are:

1) A nine-stage, reproducible pipeline that goes from the classical credit risk model to the final VaR, including en-

coding, circuit design, training, transpilation, execution and post-processing.
2) A noise-emulated backend that reproduces the three error sources above, including a hidden per-qubit angle drift that no calibration method is allowed to read.
3) An automated calibration that combines Bayesian optimisation (BO) and simultaneous perturbation stochastic approximation (SPSA). It reaches a higher fidelity than the grid sweep with half the circuit executions.
4) Readout-error mitigation and zero-noise extrapolation (ZNE) applied to the measured distributions.
5) A fidelity acceptance gate and a closed recalibration loop that detect drift and repair it automatically.
6) A timed, equal-shot comparison between the reference procedure and the proposed pipeline, run stage by stage on the same emulated hardware.

The rest of the paper is organised as follows. Section III describes the credit risk model. Section IV presents the proposed pipeline. Section V gives the experimental setup, Section VI reports the results, Section VII discusses them, and Section VIII concludes.

II. Background and Related Work

Quantum credit risk analysis. Woerner and Egger [2] showed that VaR and conditional VaR can be estimated with QAE, and Egger *et al.* [3] extended this to portfolios built on the Gaussian conditional-independence (GCI) model. In both works the uncertainty model is loaded by a dedicated circuit, and the quadratic speed-up depends on loading it accurately.

Loading distributions into qubits. Loading an arbitrary distribution exactly can require deep circuits. Variational approaches train a shallow parameterised circuit instead, for example with a quantum generative adversarial network [7] or with gradient-based training of a variational quantum circuit [8], [9]. Gradients of such circuits can be computed exactly with the parameter-shift rule [10].

Hardware adaptation. Circuits must be mapped onto the native gates and limited connectivity of a processor. The SABRE algorithm [12] is the standard routing method used in Qiskit [13]. The reference work [1] additionally retuned the gate angles on hardware by a grid sweep.

Optimisation with expensive, noisy measurements. Bayesian optimisation [14], [15] builds a probabilistic model of an unknown function and chooses each new evaluation where improvement is most likely. SPSA [17] estimates a gradient from only two measurements, whatever the number of parameters, and is widely used on quantum hardware [18].

Error mitigation. Readout errors can be corrected by measuring a calibration (assignment) matrix and inverting it [19]. Zero-noise extrapolation runs a circuit at amplified noise levels and extrapolates back to zero noise [20], [21].

III. Credit Risk Model

A. Gaussian conditional-independence model

The proposed work uses the one-factor Vasicek form of the GCI model [6]. The health of a borrower depends on a shared economic factor $z \sim \mathcal{N}(0,1)$ and on a private factor. The borrower defaults when the health falls below a threshold that is set by the average default probability p_0. Given the economy z, the default probability is

$$\mathrm{PD}(z) = \Phi\left(\frac{\Phi^{-1}(p_0) - \sqrt{\rho}\,z}{\sqrt{1-\rho}}\right), \tag{1}$$

where Φ is the standard normal cumulative distribution function and ρ is the correlation of the borrower with the economy. A negative z (a recession) raises the default probability and a positive z lowers it. The borrower is described by $p_0 = 0.25$, $\rho = 0.027$ and a loss given default (LGD) of \$1000. With these values PD rises to 0.427 at $z = -3$ and falls to 0.118 at $z = +3$ (Fig. 1). No dataset is needed: the borrower is fully described by p_0, ρ and the LGD.

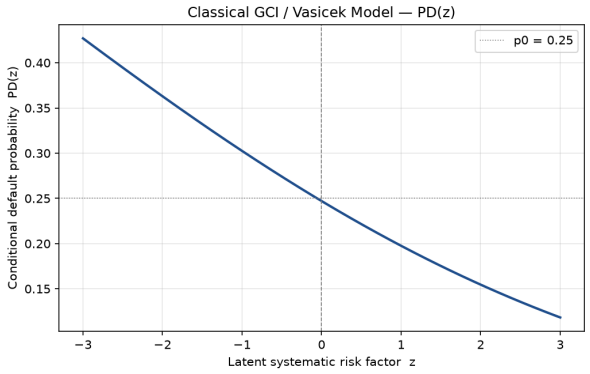


Fig. 1. Conditional default probability of the Vasicek model, Eq. (1), for $p_0 = 0.25$ and $\rho = 0.027$.

B. Discretised economy

A register of n qubits has 2^n basis states. Each state index b is mapped to one point of the economy grid, and each point receives the probability mass of the standard normal distribution:

$$z(b) = -z_{\max} + \frac{2 z_{\max}\, b}{2^n - 1}, \qquad p_b^* = \frac{e^{-z(b)^2/2}}{\sum_{b'} e^{-z(b')^2/2}}, \tag{2}$$

with $z_{\max} = 3$, which covers 99.7 % of the normal distribution. With $n = 2$ the four bins are $z \in \{-3, -1, 1, 3\}$ with masses $\{0.009, 0.491, 0.491, 0.009\}$. Fig. 2 shows the smooth classical bell curve and the slices that two and three qubits can hold.

C. Risk measures

For one borrower the loss is $L = d \cdot \mathrm{LGD}$, where $d = 1$ for default and $d = 0$ otherwise. The loss distribution gives the expected loss $\mathbb{E}[L] = \sum_L L\,P(L)$, the value at risk

$$\mathrm{VaR}_\alpha = \min\{\, L : P(\mathrm{Loss} \le L) \ge \alpha \,\}, \tag{3}$$

with $\alpha = 0.95$, and the conditional VaR (the average loss in the worst 5 % of cases). The exact classical values are a default probability of 0.25, an expected loss of \$250 and a VaR of \$1000. The same values are also obtained with a classical Monte Carlo simulation of 2 000 000 random economies,

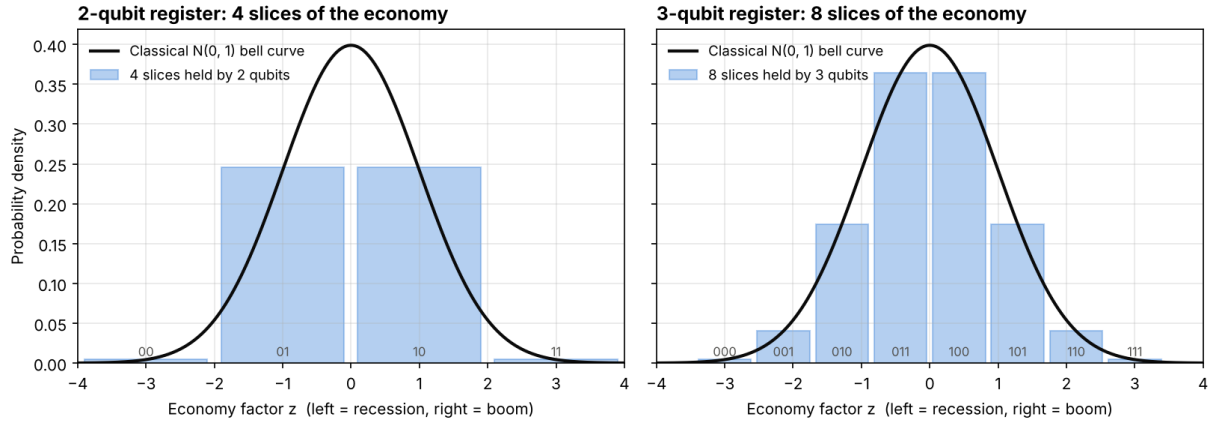


Fig. 2. Classical standard normal distribution of the economy factor z (black curve) and its discretisation into 2^n slices for a two-qubit and a three-qubit register (bars, drawn as probability density so that they share the axis with the curve). Each slice is one basis state of the register, labelled by its bitstring.

which gives a probability of zero loss of 0.7497 and the same VaR of \$1000. These classical values serve as the reference for every quantum result.

IV. Proposed Pipeline

Fig. 3 shows the complete pipeline. Stages 1 to 5 build and prepare the circuit, Stage 6 calibrates it on the noisy device, Stages 7 and 8 run it and turn the measured bitstrings into risk numbers, and Stage 9 decides whether the result can be trusted. The shaded boxes are the additions of the proposed work.

A. Stage 2: Quantum encoding

A quantum circuit reads a probability from the Born rule. A rotation $R_y(2\theta)$ applied to $|0\rangle$ gives the state $|1\rangle$ with probability $\sin^2\theta$. To store $\mathrm{PD}(z)$ in one asset qubit, it is written as

$$\mathrm{PD}(z) \approx \sin^2(\alpha z + \beta), \tag{4}$$

and α and β are found by a least-squares fit of $\arcsin\sqrt{\mathrm{PD}(z)}$ to a straight line on 400 points in $[-3, 3]$. The fit gives $\alpha = -3.47°$ and $\beta = 30.03°$, with a maximum absolute error of 0.0065 in probability. Because the register stores the index b and not z itself, Eq. (2) is substituted into Eq. (4):

$$\tilde{\alpha} = \alpha\,\frac{2z_{\max}}{2^n - 1}, \qquad \tilde{\beta} = \beta - \alpha z_{\max}, \tag{5}$$

so that $\mathrm{PD}(z(b)) \approx \sin^2(\tilde{\alpha}b + \tilde{\beta})$. This gives $\tilde{\alpha} = -6.93°$ for two qubits, $\tilde{\alpha} = -2.97°$ for three qubits, and $\tilde{\beta} = 40.42°$ in both cases.

B. Stage 3: Circuit design

The circuit has two blocks (Fig. 4).

Economy loader. Each register qubit receives an $R_y(\theta_i)$ rotation, followed by CNOT gates from qubit 0 to every other qubit (a star layout). The output distribution of the loader is

$$p_b(\boldsymbol{\theta}) = \left|\langle b \,|\, U_{\mathrm{ent}} \bigotimes_i R_y(\theta_i)\,|0\rangle^{\otimes n}\right|^2. \tag{6}$$

Asset qubit. Writing $b = \sum_k 2^k b_k$, the asset qubit receives $R_y(2\tilde{\beta})$ followed by one controlled rotation $CR_y(2^{k+1}\tilde{\alpha})$ from every register qubit k. The total rotation angle is then $2(\tilde{\alpha}b + \tilde{\beta})$, so

$$P(\text{asset} = 1 \mid b) = \sin^2(\tilde{\alpha}b + \tilde{\beta}) \approx \mathrm{PD}(z(b)). \tag{7}$$

The measured bitstring therefore contains both the economy state (register bits) and the default event (asset bit). The two-qubit register uses 3 qubits in total and the three-qubit register uses 4.

C. Stage 4: Classical training of the loader

The loader angles $\boldsymbol{\theta}$ are trained so that $p_b(\boldsymbol{\theta})$ matches the target p_b^* of Eq. (2). The training minimises the mean squared error $\mathcal{L}(\boldsymbol{\theta}) = \sum_b (p_b(\boldsymbol{\theta}) - p_b^*)^2$. Its gradient is obtained exactly with the parameter-shift rule [9], [10],

$$\frac{\partial p_b}{\partial \theta_i} = \frac{p_b(\boldsymbol{\theta} + \tfrac{\pi}{2}\mathbf{e}_i) - p_b(\boldsymbol{\theta} - \tfrac{\pi}{2}\mathbf{e}_i)}{2}, \tag{8}$$

and the angles are updated with the Adam optimiser [11] (learning rate 0.15, 150 iterations). Training runs on a noiseless simulator, because the loader distribution does not depend on the borrower and is learned once. For two qubits the training reaches $\boldsymbol{\theta}^* = [90°, 195.4°]$ with a loss of 2×10^{-8} (Fig. 5). For three qubits it reaches $[90°, 55°, 131°]$ with a loss of 0.049. The three-qubit star layout cannot represent the eight-bin Gaussian exactly; this limitation is discussed in Section VII.

D. Stage 5: Transpilation

The logical circuit is mapped to the native gate set $\{R_z, \sqrt{X}, X, CZ\}$ of a linear chain of qubits with the SABRE layout and routing method [12]. Table I shows the cost. The depth grows by a factor of about seven, and the number of two-qubit CZ gates, the main source of gate noise, reaches 8 and 14.

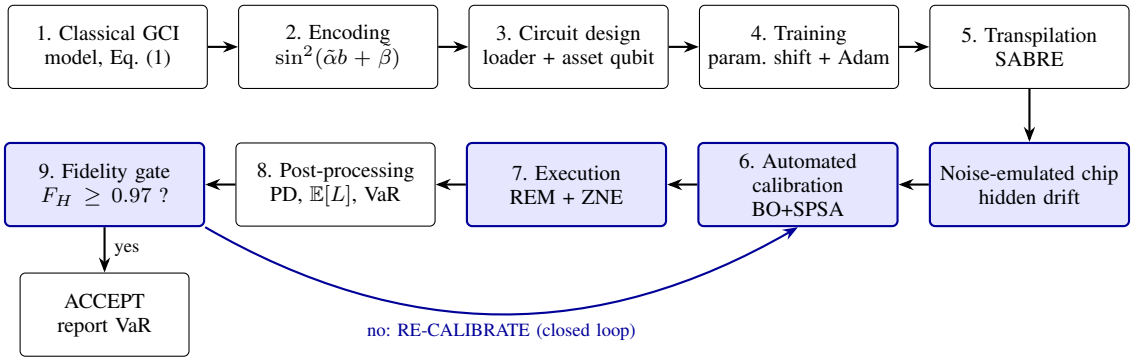


Fig. 3. Proposed pipeline. White boxes follow the standard GCI loading workflow; shaded boxes are the additions of the proposed work: the noise-emulated device with hidden drift, the automated calibration, readout-error mitigation (REM) and zero-noise extrapolation (ZNE), and the fidelity gate with a closed recalibration loop.

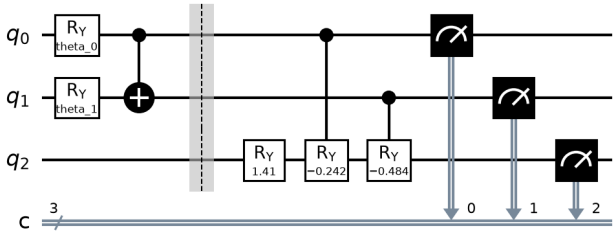


Fig. 4. Circuit for the two-qubit register: the economy loader (q_0, q_1) and the asset qubit (q_2) with one offset rotation and two controlled slope rotations.

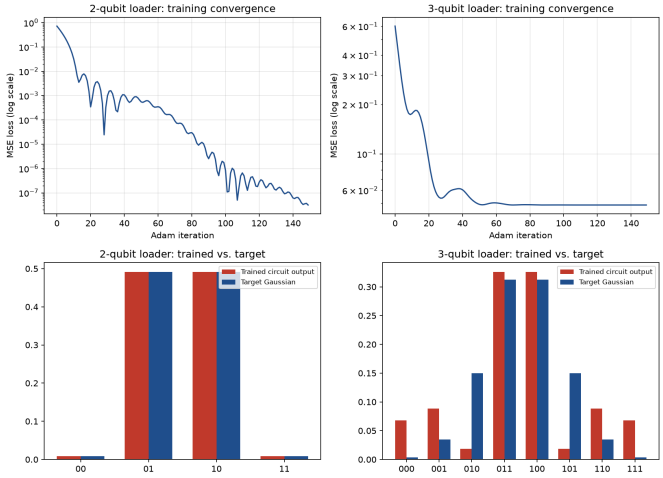


Fig. 5. Training of the economy loader with parameter-shift gradients and Adam. Top: loss per iteration. Bottom: trained output against the discretised Gaussian target.

E. Noise-emulated device

All experiments run on the Qiskit Aer simulator [13] with a noise model that contains the three error sources of a real superconducting processor:

- depolarising gate noise of 0.2 % per single-qubit gate and 1.5 % per CZ gate;
- per-qubit readout errors between 2.0 % and 7.0 %, larger for $|1\rangle \to |0\rangle$ than for $|0\rangle \to |1\rangle$;
- a hidden coherent drift on every rotation,

$$\theta_{\text{real}} = (1 + \epsilon_q)\,\theta_{\text{cmd}} + \delta_q, \tag{9}$$

with amplitude errors ϵ_q between $-11\,\%$ and $+14\,\%$ and offsets δ_q between $-15°$ and $+14°$ on physical qubit q.

For example, a commanded rotation of $90°$ on a qubit with $\epsilon = 0.12$ and $\delta = 12°$ becomes $112.8°$. No calibration method can read ϵ_q or δ_q; each method sees only measured histograms, as an experimentalist would.

TABLE I
CIRCUIT SIZE BEFORE AND AFTER TRANSPILATION

	Two-qubit register		Three-qubit register	
	Logical	Native	Logical	Native
Qubits	3	4	4	5
Depth	6	33	8	56
Two-qubit gates	3	8	5	14
Single-qubit gates	3	34	4	56

F. Stage 6: Automated calibration

Objective. Calibration looks for the commanded angles $\mathbf{x} = [\boldsymbol{\theta}, \tilde{\beta}, \tilde{\alpha}]$ that make the device output $\mathbf{p}(\mathbf{x})$ match the ideal output $\mathbf{q}$ of the same circuit on a perfect device. The mismatch is measured with the Hellinger distance

$$J(\mathbf{x}) = \sqrt{1 - \textstyle\sum_i \sqrt{p_i(\mathbf{x})\, q_i}}, \qquad F_H = \Big(\textstyle\sum_i \sqrt{p_i q_i}\Big)^2, \tag{10}$$

where F_H is the Hellinger fidelity. $J = 0$ (or $F_H = 1$) means a perfect match. Every evaluation of J is one circuit execution of 4000 shots on readout-mitigated data, so that the calibration corrects only what readout mitigation cannot.

Reference procedure (grid sweep). The reference procedure evaluates J on a coarse grid, keeps the best point and repeats on a fine grid around it, for one angle or one pair of angles at a time. With the published ranges and steps this costs 347 executions for two qubits and 477 for three.

Proposed procedure. The proposed calibration has four steps (Algorithm 1).

1) **BO on the loader angles** (40 executions). A Gaussian process with a radial basis function kernel [16] models J inside a box of $\pm 45^\circ$ around the trained angles. After a small space-filling initial design, each new point maximises the expected improvement [14]

$$\mathrm{EI}(\mathbf{u}) = I\,\Phi\Big(\tfrac{I}{s(\mathbf{u})}\Big) + s(\mathbf{u})\,\varphi\Big(\tfrac{I}{s(\mathbf{u})}\Big), \qquad (11)$$

where $I = J_{\min} - \mu(\mathbf{u}) - \xi$, μ and s are the predicted mean and standard deviation, φ is the standard normal density and ξ is a small exploration margin. BO therefore spends executions only where improvement is likely and skips poor regions.

2) **BO on the two asset angles** (30 executions), with the loader angles fixed at the result of step 1.

3) **SPSA on all angles together** (128 executions). SPSA [17] perturbs every angle at once by $\pm c_k$ along a random sign vector $\boldsymbol{\Delta}_k$ and estimates the gradient from two executions,

$$\hat{\mathbf{g}}_k = \frac{J(\mathbf{x}_k + c_k\boldsymbol{\Delta}_k) - J(\mathbf{x}_k - c_k\boldsymbol{\Delta}_k)}{2c_k}\,\boldsymbol{\Delta}_k, \qquad \mathbf{x}_{k+1} = \mathbf{x}_k - a_k\hat{\mathbf{g}}_k, \qquad (12)$$

with the standard gains $a_k = a/(k+1+A)^{0.602}$ and $c_k = c/(k+1)^{0.101}$. The method runs 60 iterations plus 4 pairs that set the step size, and returns the average of the last quarter of the iterates to suppress shot noise. Because all angles move together, this step corrects the coupling between the loader and the asset qubit that a one-angle-at-a-time search cannot see.

4) **Verification** (6 executions). The results of steps 2 and 3 are each re-measured three times and the better one is kept.

The total cost is a fixed 204 executions per register, independent of the register size.

G. Stage 7: Execution and error mitigation

The calibrated circuit is executed 20 times with 20 000 shots each, so that every result carries an error bar.

Readout-error mitigation (REM). Each basis state is prepared and measured once to build the assignment matrix A, whose entry A_{ij} is the probability of reading i when j was prepared [19]. The measured distribution satisfies $\mathbf{p}_{\mathrm{meas}} = A\,\mathbf{p}_{\mathrm{true}}$, and the true distribution is recovered as

$$\hat{\mathbf{p}} = \arg\min_{\mathbf{p}\ge 0} \|A\mathbf{p} - \mathbf{p}_{\mathrm{meas}}\|_2, \qquad (13)$$

by non-negative least squares, followed by normalisation.

Zero-noise extrapolation (ZNE). The circuit U is replaced by $U(U^\dagger U)^k$, which is logically identical but carries $\lambda = 2k+1$ times more gate noise [21]. The circuit is measured at $\lambda \in \{1,3,5\}$, each outcome probability is fitted with a straight line in λ, and the line is evaluated at $\lambda = 0$ [20]. The $\lambda = 1$ point reuses the ordinary execution, so ZNE adds only the $\lambda = 3$ and $\lambda = 5$ runs.

Algorithm 1 Automated calibration with closed-loop recalibration

Input: trained angles $\mathbf{x}_0$, ideal output $\mathbf{q}$, threshold $F_{\min} = 0.97$
Output: accepted angles $\mathbf{x}^*$ and risk measures

measure the readout assignment matrix A
$\boldsymbol{\theta} \leftarrow$ BO on the loader angles (40 runs)
$(\tilde{\beta}, \tilde{\alpha}) \leftarrow$ BO on the asset angles (30 runs)
$\mathbf{x}_{\mathrm{SPSA}} \leftarrow$ SPSA on all angles (128 runs)
$\mathbf{x}^* \leftarrow$ better of BO and SPSA results (6 runs)
execute $\mathbf{x}^*$, correct readout with A, optionally apply ZNE
compute F_H against $\mathbf{q}$
if $F_H \ge F_{\min}$ **then**
    ACCEPT: compute PD, $\mathbb{E}[L]$, VaR and report
else
    RE-CALIBRATE: repeat lines 1–7 starting from $\mathbf{x}^*$ (at most 3 rounds)
end if

H. Stage 8: Post-processing

In each measured bitstring the left bit is the default flag d and the remaining bits give the economy bin b. The joint distribution $p(d,b)$ gives the conditional default probability

$$\widehat{\mathrm{PD}}(z_b) = \frac{p(1,b)}{p(0,b) + p(1,b)}, \qquad (14)$$

the total default probability $\sum_b p(1,b)$, the loss distribution, the expected loss, the VaR of Eq. (3) and the conditional VaR.

I. Stage 9: Validation, fidelity gate and closed loop

Four validation checks. A single fidelity number cannot tell a hardware error from a modelling error, so every result is checked in four ways:

1) fidelity of the measured distribution against the ideal output of the same circuit (hardware quality);
2) fidelity against the classical joint distribution of default and economy (model quality);
3) fidelity of the loss distribution against the classical loss distribution (risk quality);
4) fidelity of the economy marginal $P(z_b)$ against the target Gaussian (loader quality).

The first check drives the acceptance decision; the other three show where any remaining error comes from.

The deployed result is accepted only if $F_H \ge 0.97$ against the ideal output. Otherwise it is marked RE-CALIBRATE and the automated calibration is run again on the current device, starting from the last accepted angles (Algorithm 1). In this way a wrong quantum result never reaches the risk report, and drift is repaired without manual work.

V. Experimental Setup

The pipeline is written in Python with Qiskit and Qiskit Aer [13], NumPy and SciPy. Each stage is a separate notebook backed by shared modules, and a master script runs all nine stages in order. Table II lists the main settings.

TABLE II
SIMULATION SETTINGS

Parameter	Value
Borrower	$p_0 = 0.25$, $\rho = 0.027$, LGD = \$1000
Economy grid	$z \in [-3, 3]$, 4 or 8 bins
Training	Adam, learning rate 0.15, 150 iterations
Native gates	R_z, $\sqrt{X}$, X, CZ, linear chain
Gate noise	0.2 % (1-qubit), 1.5 % (2-qubit)
Readout error	2.0 % to 7.0 % per qubit
Hidden drift	$\lvert\epsilon_q\rvert \leq 14\,\%$, $\lvert\delta_q\rvert \leq 15^\circ$
Calibration shots	4000 per execution
Execution	20 repeats $\times$ 20 000 shots
BO budget	40 (loader) + 30 (asset), box $\pm 45^\circ$
SPSA	60 iterations, 128 executions
ZNE scale factors	1, 3, 5 (linear fit)
Acceptance threshold	$F_H \geq 0.97$, at most 3 rounds

Equal-shot comparison. To compare the reference procedure with the proposed pipeline fairly, both were rebuilt as separate pipelines that share Stages 1 to 5 and use the same emulated device, the same inputs and the same number of shots for every execution (4000 per calibration execution, 20 repeats of 20 000 shots for execution and 20 000 for each drift check). The reference pipeline uses the grid sweep, no readout mitigation, no acceptance gate, and repeats the sweep after a drift. The proposed pipeline uses the automated calibration, REM, ZNE with five repeats, the acceptance gate and the closed loop. Every stage starts and stops a timer, and independent circuits are sent to the simulator together in one call. All wall-clock times were measured from the terminal on the same laptop.

Drift event. To test the closed loop, the device is calibrated and then its hidden drift is replaced by a new, different set of ϵ_q and δ_q, as if the processor had been recalibrated overnight.

VI. RESULTS

A. Encoding and training

For the two-qubit register the noiseless circuit gives conditional default probabilities of 0.420, 0.304, 0.200 and 0.113 for the four economy bins, against the exact Vasicek values of 0.427, 0.303, 0.198 and 0.118. The total default probability is 0.2525 against 0.2504. The encoding and training stages are therefore accurate, and every error seen later comes from the hardware.

B. Calibration quality and cost

Table III and Fig. 6 compare the calibration methods on the full circuit. Without calibration, the trained angles give a fidelity of only 0.880 and 0.920 on the noisy device. The grid sweep raises this to 0.939 and 0.957 with 347 and 477 executions, which stays below the 0.97 threshold. The automated calibration reaches 0.982 and 0.994 with 204 executions in both cases.

Two ablations explain why the proposed method is built in steps. Joint SPSA alone on all angles reaches 0.982 for two qubits but only 0.947 for three qubits after 308 executions. Joint BO alone reaches 0.973 and 0.964 with 70 executions. Using BO first to find the right region and SPSA afterwards to refine all angles together gives the best result at the lowest cost. On the loader circuit alone the difference is even clearer: BO reaches a fidelity of 0.9997 and 0.9998 with 40 executions, while the grid sweep reaches 0.962 with 61 executions (two qubits) and 0.965 with 191 executions (three qubits).

TABLE III
CALIBRATION OF THE FULL CIRCUIT ON THE NOISY DEVICE (HELLINGER FIDELITY MEASURED WITH 20 000 SHOTS)

	Two-qubit		Three-qubit	
Method	Runs	F_H	Runs	F_H
Uncalibrated (raw)	0	0.880	0	0.920
Grid sweep (reference)	347	0.939	477	0.957
Joint BO only (ablation)	70	0.973	70	0.964
Joint SPSA only (ablation)	308	0.982	308	0.947
Automated BO+SPSA	**204**	**0.982**	**204**	**0.994**

TABLE IV
FIDELITY AFTER EACH CORRECTION (MEAN $\pm$ STANDARD DEVIATION OVER 20 REPEATS)

Variant	Two-qubit	Three-qubit
Uncalibrated, raw	0.879 ± 0.003	0.919 ± 0.002
Uncalibrated + REM	0.922 ± 0.003	0.909 ± 0.003
Grid sweep + REM	0.939 ± 0.002	0.956 ± 0.002
Automated + REM	0.982 ± 0.002	0.995 ± 0.001
Automated + REM + ZNE	$\mathbf{0.998 \pm 0.001}$	$\mathbf{0.996 \pm 0.001}$

C. Effect of each correction

Table IV shows the fidelity of the executed circuit, as the mean and standard deviation over 20 repeats. Readout mitigation alone cannot repair the drift: for three qubits it even lowers the fidelity slightly, from 0.919 to 0.909, because the remaining error is coherent and not a readout error. Calibration is therefore essential. The automated calibration with REM reaches 0.982 and 0.995, and adding ZNE removes most of the remaining gate noise, giving 0.998 and 0.996. For orientation, the hardware experiment in [1] reported a fidelity of 0.989; a direct comparison is not possible because the device in the proposed work is emulated.

D. Credit risk outputs

Table V lists the risk measures. Every variant gives the correct VaR of \$1000, because the probability of zero loss stays below 0.95. The expected loss is more sensitive. The uncalibrated circuit underestimates it strongly (\$174 against \$250 for two qubits) because the drift lowers the default rate. The grid sweep reaches \$245 and the automated calibration reaches \$273, and ZNE brings it to \$258. Fig. 7 shows the conditional default probability per economy state. The calibrated results follow the Vasicek curve in the central bins. The extreme bin $z = +3$ carries only 0.9 % of the probability for two qubits, so it receives very few shots and remains the noisiest point. The other validation checks confirm that the risk output is reliable: after automated calibration with REM, the loss distribution matches the classical one with a fidelity of 0.9993 (two qubits) and 0.9998 (three qubits), and the

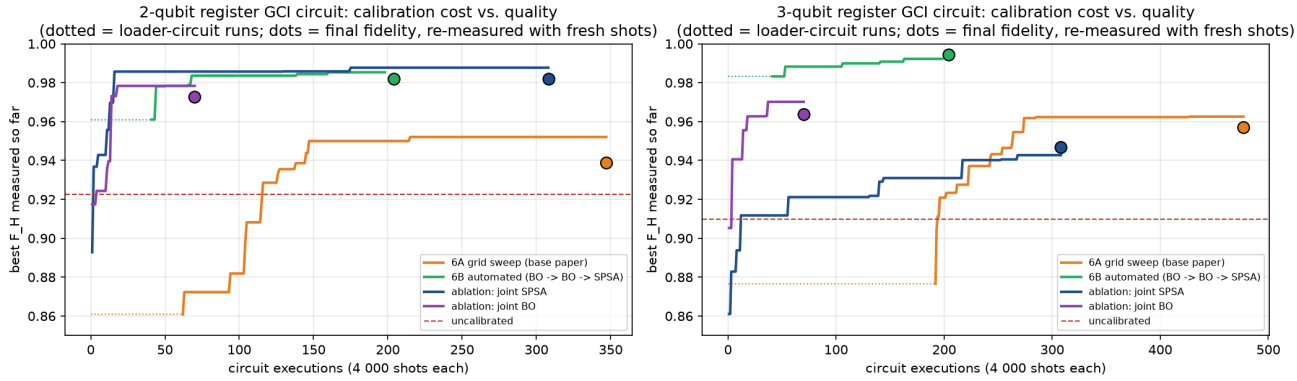


Fig. 6. Calibration cost against quality. Each curve shows the best fidelity found so far against the number of circuit executions; dots show the final fidelity re-measured with fresh shots. The automated method (green) passes the grid sweep (orange) early and ends higher with fewer executions.

TABLE V
CREDIT RISK MEASURES FOR THE DEFAULT BORROWER

	Two-qubit		Three-qubit	
Source	PD	$\mathbb{E}[L]$ ($)	PD	$\mathbb{E}[L]$ ($)
Classical (exact)	0.250	250.4	0.250	250.0
Uncalibrated, raw	0.174	174.1	0.191	190.7
Grid sweep + REM	0.245	244.7	0.263	262.5
Automated + REM	0.273	272.9	0.261	261.3
Automated + REM + ZNE	0.258	258.1	0.230	229.9
VaR at 95 % is $1000 in every case, equal to the classical value.				

economy marginal matches the target Gaussian with a fidelity of 0.984 for two qubits.

E. Drift detection and closed-loop recovery

Fig. 8 shows the closed loop. On the calibrated device the deployed pipeline is accepted with $F_H = 0.982$ and 0.994. After the drift event the same angles give only 0.528 and 0.676, so the gate returns RE-CALIBRATE. One automatic round of the calibration (204 executions) restores the fidelity to 0.972 and 0.992, and the result is accepted again. No manual step is needed.

F. Reference procedure against proposed pipeline

Table VI summarises the timed, equal-shot comparison, and Fig. 9 shows the cost of each part. These numbers come from the two separate equal-shot pipelines, so they differ slightly from the main pipeline above.

Circuit executions. The proposed pipeline needs 952 executions against 1692, which is 44 % fewer, even though it also performs readout calibration and ZNE. Calibration alone needs 408 executions against 824 (50 % fewer), and drift recovery needs 448 against 828 (46 % fewer).

Quality. Without readout mitigation the reference procedure reaches 0.900 and 0.948, and after the drift it reaches only 0.900 and 0.957, which stays below the acceptance threshold. The proposed pipeline reaches 0.983 and 0.992, 0.998 and 0.997 with ZNE, and recovers to 0.982 and 0.985 after the drift.

TABLE VI
EQUAL-SHOT COMPARISON OF THE REFERENCE PROCEDURE AND THE PROPOSED PIPELINE

Quantity	Reference	Proposed
Readout calibration runs	–	36
Calibration runs	824	408
Execution runs	40	40
ZNE runs	–	20
Drift recovery runs	828	448
Total circuit runs	**1692**	**952**
F_H after calibration (2q / 3q)	0.900 / 0.948	0.983 / 0.992
F_H with ZNE (2q / 3q)	–	0.998 / 0.997
F_H after drift (2q / 3q)	0.900 / 0.957	0.982 / 0.985
Acceptance at $F_H \geq 0.97$	no	yes
VaR at 95 %	$1000	$1000
Laptop wall-clock, run 1 (s)	12.93	11.52
Laptop wall-clock, run 2 (s)	11.60	11.50
Estimated chip time (s)	741	477

Wall-clock time. In two independent runs on a laptop, the full proposed pipeline took 11.52 s and 11.50 s, against 12.93 s and 11.60 s for the reference procedure. The proposed pipeline is therefore as fast or faster, while doing more work and reaching a higher fidelity.

Why the wall-clock difference is small. Table VII breaks the time down by stage for one run on a second machine (all values in seconds; this machine is slower than the laptop, but the pattern is the same). Halving the number of executions saves time in the two stages that calibrate: calibration is 22 % faster (6.77 s against 8.72 s) and drift recovery is 24 % faster (6.60 s against 8.63 s). The saving is smaller than the 50 % cut in executions for two reasons. First, Bayesian optimisation must fit its Gaussian process model and search for the next point after every execution, and it must wait for each result before choosing the next angles, so its executions are sent one after another instead of in a few large batches as in the grid sweep. On a simulator, where one execution takes only milliseconds, this per-step overhead is a large share of the cost. Second, part of the time saved is spent on steps that the reference procedure does not have: the readout calibration (0.38 s), zero-

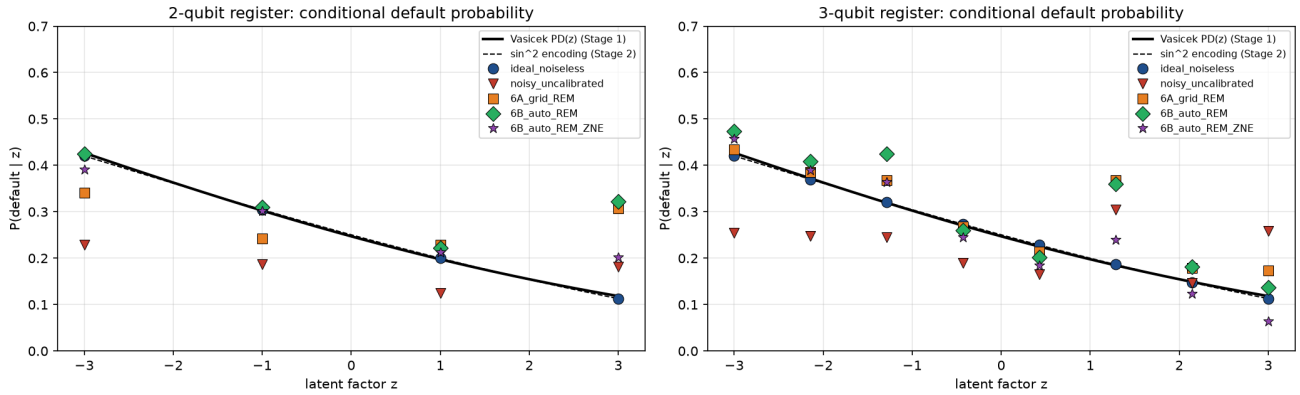


Fig. 7. Conditional default probability per economy state, Eq. (14), against the Vasicek curve (solid) and the $\sin^2$ encoding (dashed).

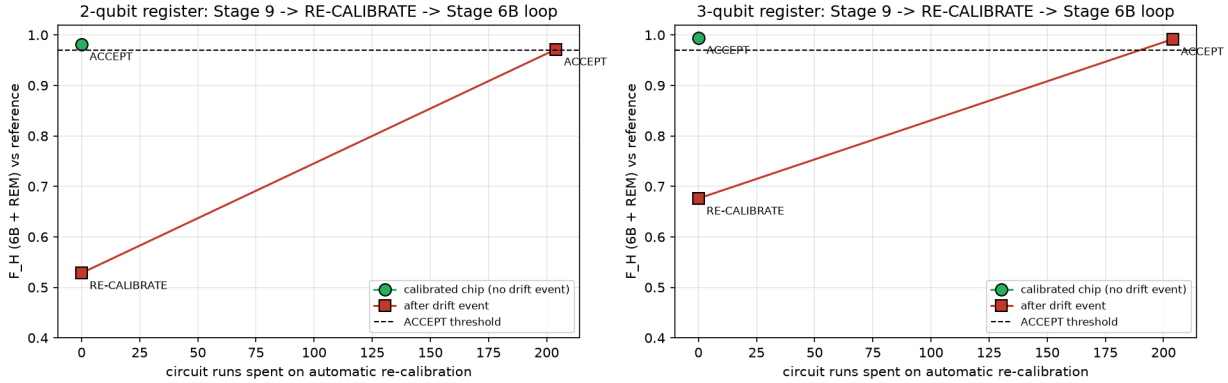


Fig. 8. Closed-loop recalibration. Green: calibrated device, accepted. Red: after a drift event the fidelity falls below the threshold (dashed); one automatic round of calibration brings it back above the threshold.

TABLE VII
WALL-CLOCK TIME PER STAGE FOR ONE RUN OF EACH PIPELINE (SECONDS, EQUAL SHOTS)

Stage	Reference	Proposed
Stages 1–5 and other steps	1.36	1.20
Readout setup (add-on)	–	0.38
Calibration	8.72	6.77
Execution	1.76	2.27
ZNE (add-on)	–	1.47
Drift recovery	8.63	6.60
Total	**20.47**	**18.70**

noise extrapolation (1.47 s) and readout correction of every execution. The net result is a total that is equal to or slightly below the reference, while the proposed pipeline does more and reaches a much higher fidelity. On a real processor the per-step overhead of Bayesian optimisation (milliseconds) is negligible next to the physical time of an execution (about 0.4 s for 4000 shots), so the time follows the number of executions.

Estimated time on a quantum processor. On real hardware every execution is a physical experiment. Assuming 100 μs per shot, which covers qubit reset, gates and readout, the shots of Table VI correspond to about 741 s of processor time for the reference procedure and 477 s for the proposed pipeline. For calibration alone the estimate is 330 s against 163 s.

VII. Discussion

Why fewer executions do not always mean fewer seconds on a simulator. On a simulator a single execution takes only milliseconds, so the wall-clock time is dominated by software overhead around each call. The grid sweep knows all its points in advance and sends them to the simulator in a few large batches. The automated calibration must see each result before choosing the next point, so it makes many smaller calls and also spends a little time updating its Gaussian process model. This is why the laptop times are close even though the proposed method uses half the executions. On a real processor the cost of each execution is fixed by physics and cannot be batched away, so the number of executions decides the time, and the proposed method is about twice as fast for calibration.

What is minimised. Both methods minimise the same objective, the Hellinger distance of Eq. (10). The proposed method reaches a lower final value, about 0.09 and 0.06 for the two registers against 0.23 and 0.16 for the reference procedure without readout mitigation, with fewer executions.

Role of each addition. Calibration corrects coherent angle errors, REM corrects measurement errors and ZNE reduces

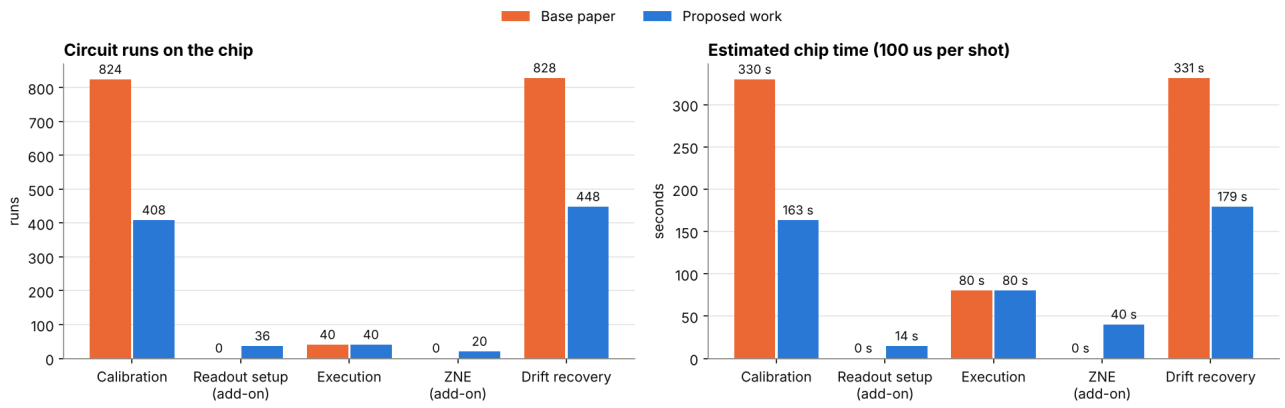


Fig. 9. Circuit executions (left) and estimated processor time at 100 μs per shot (right) for each part of the reference procedure and the proposed pipeline, at equal shots per execution.

incoherent gate noise. They address different error sources, and the results show that each adds fidelity that the others cannot provide. The acceptance gate and the closed loop turn these corrections into a system that can run unattended.

Why quantum. For a single borrower a classical computer computes the answer exactly, and the proposed work uses it to verify every quantum result. The quantum advantage appears at portfolio scale, where QAE can estimate the VaR with quadratically fewer samples than Monte Carlo [2], [3]. QAE, however, requires an accurate loading circuit. The proposed work makes that circuit reliable on noisy hardware, which is the step needed before the speed-up can be used.

Limitations. The device is emulated, so a run on a real processor remains to be done. The three-qubit star layout cannot represent the eight-bin Gaussian exactly (training loss 0.049), so its fidelity against the classical distribution is about 0.81 even when the hardware fidelity is 0.99; a chain layout reaches a training loss of 0.0018 and is a direct improvement. For borrowers with a low default probability (around 4 %) the small absolute error of the circuit can push the default probability across 5 % and change the VaR. Finally, QAE itself is not yet implemented.

VIII. Conclusion and Future Work

The proposed work builds a complete, hardware-aware pipeline that loads a credit risk model into a variational quantum circuit and computes the default probability, expected loss and value at risk on a noise-emulated processor. The manual grid sweep for retuning the gates was replaced by an automated calibration that combines Bayesian optimisation and SPSA. Together with readout-error mitigation, zero-noise extrapolation, a fidelity acceptance gate and a closed recalibration loop, the pipeline reaches a Hellinger fidelity of 0.98 to 0.99 (0.996 to 0.998 with ZNE) and the correct VaR of \$1000. It halves the calibration cost, needs 44 % fewer circuit executions overall, and recovers automatically from calibration drift, while running in the same or less wall-clock time than the reference procedure.

Future work will run the pipeline on a real superconducting processor, replace the three-qubit star loader with a chain layout, extend the model to several borrowers and economic factors, and add quantum amplitude estimation on top of the calibrated loading circuit.

Acknowledgment

The authors thank Dr. Snigdhatanu Acharya, School of Artificial Intelligence, Amrita Vishwa Vidyapeetham, for guidance throughout this project.

References

[1] H. G. Ahmad *et al.*, "Quantum circuit-based adaptation for credit risk analysis," *IEEE Trans. Quantum Eng.*, vol. 7, Art. no. 3103316, 2026, doi: 10.1109/TQE.2026.3691176.

[2] S. Woerner and D. J. Egger, "Quantum risk analysis," *npj Quantum Inf.*, vol. 5, Art. no. 15, 2019.

[3] D. J. Egger, R. García Gutiérrez, J. Cahué Mestre, and S. Woerner, "Credit risk analysis using quantum computers," *IEEE Trans. Comput.*, vol. 70, no. 12, pp. 2136–2145, 2021.

[4] G. Brassard, P. Høyer, M. Mosca, and A. Tapp, "Quantum amplitude amplification and estimation," *Contemp. Math.*, vol. 305, pp. 53–74, 2002.

[5] J. Preskill, "Quantum computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, 2018.

[6] O. Vasicek, "The distribution of loan portfolio value," *Risk*, vol. 15, no. 12, pp. 160–162, 2002.

[7] C. Zoufal, A. Lucchi, and S. Woerner, "Quantum generative adversarial networks for learning and loading random distributions," *npj Quantum Inf.*, vol. 5, Art. no. 103, 2019.

[8] M. Cerezo *et al.*, "Variational quantum algorithms," *Nat. Rev. Phys.*, vol. 3, pp. 625–644, 2021.

[9] K. Mitarai, M. Negoro, M. Kitagawa, and K. Fujii, "Quantum circuit learning," *Phys. Rev. A*, vol. 98, Art. no. 032309, 2018.

[10] M. Schuld, V. Bergholm, C. Gogolin, J. Izaac, and N. Killoran, "Evaluating analytic gradients on quantum hardware," *Phys. Rev. A*, vol. 99, Art. no. 032331, 2019.

[11] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.

[12] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for NISQ-era quantum devices," in *Proc. ASPLOS*, 2019, pp. 1001–1014.

[13] A. Javadi-Abhari *et al.*, "Quantum computing with Qiskit," arXiv:2405.08810, 2024.

[14] D. R. Jones, M. Schonlau, and W. J. Welch, "Efficient global optimization of expensive black-box functions," *J. Global Optim.*, vol. 13, pp. 455–492, 1998.

[15] J. Snoek, H. Larochelle, and R. P. Adams, "Practical Bayesian optimization of machine learning algorithms," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2012, pp. 2951–2959.
[16] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*. Cambridge, MA, USA: MIT Press, 2006.
[17] J. C. Spall, "Multivariate stochastic approximation using a simultaneous perturbation gradient approximation," *IEEE Trans. Autom. Control*, vol. 37, no. 3, pp. 332–341, 1992.
[18] A. Kandala *et al.*, "Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets," *Nature*, vol. 549, pp. 242–246, 2017.
[19] S. Bravyi, S. Sheldon, A. Kandala, D. C. McKay, and J. M. Gambetta, "Mitigating measurement errors in multiqubit experiments," *Phys. Rev. A*, vol. 103, Art. no. 042605, 2021.
[20] K. Temme, S. Bravyi, and J. M. Gambetta, "Error mitigation for short-depth quantum circuits," *Phys. Rev. Lett.*, vol. 119, Art. no. 180509, 2017.
[21] T. Giurgica-Tiron, Y. Hindy, R. LaRose, A. Mari, and W. J. Zeng, "Digital zero noise extrapolation for quantum error mitigation," in *Proc. IEEE Int. Conf. Quantum Comput. Eng. (QCE)*, 2020, pp. 306–316.